



NotebookLab

**Every tool that reads your documents keeps a copy.
This one keeps it on your disk.**

Your thinking partner, on your machine. A desktop notebook that reads your PDFs, notes and scans, then answers with the page it used. No account, no key, nothing uploaded.

5

formats read

3

platforms

0

accounts needed

MIT

licensed

START HERE

What this is, in one page

NotebookLab imports your documents, indexes them locally, and answers questions about them using a model that runs on the same machine. Every answer names the document, the heading and the page it drew from, so you can check it in seconds rather than trust it.

It is a desktop application, not a service. There is no account to create, no key to paste, and no upload step. Once a model is downloaded it works with the network cable out.

WHO IT IS FOR

Anyone whose documents are not allowed to leave: researchers with unpublished work, teams under a compliance rule, students with a term of scanned handouts, engineers with internal runbooks.

WHAT IT COSTS

Nothing. It is MIT licensed, there is no paid tier, and there is nothing to sign up for. The only money involved is your own electricity.

WHAT IT IS NOT

- Not a chat wrapper. Retrieval and citation are the product; the model is swappable.
- Not a cloud service with a local mode. There is no server component at all.
- Not a replacement for a hosted tool if your files are free to travel. If they are, use whatever is easiest.

THE HONEST LIMITS

A local model is slower than a hosted one and weaker at long synthesis across many documents. It knows nothing about the world past its training data. Cloud providers are supported for exactly those cases, and are off until you add a key.

WHY WE BUILT IT

The tools we liked kept moving online, behind accounts and subscriptions, where every note travels through someone else's server. We wanted a place to think that belongs to nobody but the person thinking.



Why the two of us built this

Amey Thakur

I kept a folder of papers I could not put into any tool I liked. Not because they were secret, but because uploading someone else's unpublished work to a service I do not control is a decision I did not want to keep making at two in the morning.

So the rule for this project was simple: if a feature needs your documents to leave the machine, it does not ship. That rule cost us things. It is also the only reason the app is worth anything.

The part I am proudest of is not the model work. It is that every answer tells you which page it came from, so you can disagree with it.

Archit Konde

Most of what I did here is invisible when it works: the parsers that survive a malformed PDF, the chunk that remembers where it came from, the checks that stop a bad file taking the whole app down.

I care about that because a tool you cannot trust with a strange file is a tool you stop reaching for. We wrote tests for the failures that return something plausible, not just the ones that crash, because those are the expensive ones.

This is the first thing we have released publicly. We would rather hear that it broke on your files than never hear from you at all.



Everything it does, on one page



Everything it does, on your own machine

Reads your documents, answers with the page it used. Free, open source, no account.

READS

PDF

including scanned pages

Word

.docx

Markdown and text

any plain file

Images

read locally by OCR

Drag and drop

or Open with

ANSWERS WITH

The document

which file it came from

The heading

where inside it

The page

so you can check it

The passage

one click away

Search

full text and semantic

RUNS ON

Bundled llama.cpp

any GGUF model

Ollama

if you already use it

OpenAI-compatible

any endpoint

Anthropic, Gemini

your own key

Nothing at all

works offline

ALSO DOES

Connected notes

with a graph

Document outline

jump anywhere

Prompt Studio

build and reuse

Export

Word and RTF

Command palette

Ctrl+K

Windows, macOS and Linux · updates itself, every release signed · MIT licensed · no account, no telemetry, nothing uploaded



Amey Thakur with Archit Konde

github.com/Amey-Thakur/NotebookLab

Your thinking partner, on your machine.



Amey Thakur with Archit Konde

github.com/Amey-Thakur/NotebookLab

THE IDEA

An answer without a source is a rumour with good grammar

The hard part of local AI is not the model, it is the retrieval. A small model that cites its source is more useful than a large one that does not, because you can check it in seconds.

So provenance is built into the shape of the data rather than added at the end. Every format parses down to the same chunk: PDF, Word, Markdown, plain text, and images through OCR. That chunk carries which file it came from, which heading it sat under, and which page it was on.

A chunk that does not know its own origin is never created. Retrieval therefore never has to reconstruct where something came from, and no shortcut later can quietly lose it.

WHAT A CITATION GIVES YOU

FIELD	WHAT IT IS FOR
Document	Which file the claim came from
Heading	Where inside that file
Page	So you can open it and read the passage
Passage	One click, the exact text used

Deciding this shape before writing a line of retrieval was the choice that saved the most time on the project. Every shortcut that looked tempting later would have cost provenance, and provenance is the product.



Four people who cannot send documents away



Four people who cannot send documents away

The constraint is the reason this exists. Each of these is a real workflow.

THE RESEARCHER

Sixty papers

one literature review

Ask across all of them

not one at a time

Every claim cited

paper, section, page

Quote it directly

the citation is written

Nothing uploaded

preprints stay yours

LEGAL AND COMPLIANCE

Contracts

that cannot leave

Ask the bundle

not each file

Check the clause

it quotes the source

No network needed

air-gapped if you like

No third party

nothing to review

THE STUDENT

Lecture notes

a whole term

Scanned handouts

OCR reads them

Ask a concept

it points at the slide

Search everything

typed and photographed

Free

no subscription

THE ENGINEER

Design docs

and runbooks

Ask mid-incident

no tab hunting

Get the section

not a paraphrase

Stays on the laptop

internal is internal

Works offline

even on a plane

If your documents are allowed to leave, use whatever is easiest. This is for when they are not.



Amey Thakur with Archit Konde
github.com/Amey-Thakur/NotebookLab

Your thinking partner, on your machine.



Where your documents actually go



Where your documents actually go

Every tool that reads your files needs a copy. The question is whose disk it is on.

YOUR MACHINE

- **Your documents**

PDF, Word, notes, scans, imported once

- **The index**

chunks that remember file, heading and page

- **The model**

runs on your own CPU or GPU

- **The answer**

with the passage it used attached

A cloud model

Optional, and off unless you add your own key. Turn it on and your text does go to that provider, which is exactly why it is a choice, not a default.

With a local model there is no request to make, so there is nothing to intercept.



Amey Thakur with Archit Konde
github.com/Amey-Thakur/NotebookLab

Your thinking partner, on your machine.



What the privacy claim costs



What “nothing leaves your machine” costs

One sentence on a landing page. Six decisions somebody has to keep making.

LOCAL API

Binds 127.0.0.1 and nothing else

AUTH TOKEN

Regenerated every launch, never written to disk

INFERENCE SIDECAR

Pinned to one release, SHA256 verified before it runs

MODEL DOWNLOADS

Restricted to HTTPS huggingface.co, in code not in docs

WEBVIEW CSP

Allows the app’s own origin, so a document cannot call home

SQL

Parameterized everywhere, because the files came from elsewhere

The claim is one line in a README. The property is six files, and it has to be chosen again every time.



Amey Thakur with Archit Konde
github.com/Amey-Thakur/NotebookLab

Your thinking partner, on your machine.



GETTING STARTED

From download to first answer

1. INSTALL

Download the build for your platform from the releases page. Windows gets an installer and an msi, macOS a dmg for both architectures, Linux a deb and an rpm. Every artifact is signed, and the app verifies an update before it installs it.

macOS is not yet notarised, so Gatekeeper will ask. Right-click, then Open, and it will let you through.

2. PICK A MODEL

On first run it suggests a local model sized to your machine. On a laptop with 16 GB and no GPU, something in the 3 to 4B instruct range answers while you are still watching. Reasoning models are worth the wait only when you can give them a GPU.

3. IMPORT

Use the picker, or open a file with NotebookLab from your file manager. Scanned pages go through OCR on the way in, so a photographed handout is searchable alongside everything typed.

4. ASK

Ask a question whose answer is a specific claim rather than a summary. Specific questions produce specific citations, which is where the tool earns its keep.

IF SOMETHING IS SLOW

A local model on a CPU is thinking, not hung. Pick a smaller one in Models, or add a cloud key and let the two share the work.



UNDER THE HOOD

How it is put together

LAYER	WHAT IT IS
Shell	Tauri v2, so the window is the platform's own webview
Backend	Rust: parsing, OCR, retrieval, storage
Frontend	React 19 and TypeScript
Store	SQLite with WAL and FTS5
Local model	A bundled llama.cpp sidecar, any GGUF
OCR	ocrs on the pure-Rust rten runtime, no system dependency
Commands are async Tauri IPC into services, then repositories for data, providers for models, and parsers for formats. Each module is self-contained enough to remove.	

NO VECTOR DATABASE

Embeddings live in SQLite and similarity is brute-force cosine in Rust. Over a few thousand chunks that is fast enough that a separate database would be an operational liability rather than an upgrade. Add one when the numbers say so, not before.

EVERYTHING IS CHECKABLE

The repository is MIT licensed and every claim in this booklet is implemented in code you can read. The security posture in particular is written down file by file, so it can be audited rather than believed.



THE END

Take it, fork it, tell us what breaks

NotebookLab is the first software the two of us have released publicly. We built the notebook we wanted: one that reads your documents, answers your questions, and never sends a word off your machine.

It is open in full, including the parts we got wrong first. If it fails on your files, an issue with the file type and what happened is worth more to us than a star.

WHERE TO GO

- github.com/Amey-Thakur/NotebookLab
- amey-thakur.github.io/NotebookLab

THE PLEDGE

No account. No telemetry. No analytics. Nothing phones home. Your documents stay on your disk, and the only way text ever leaves is if you add a cloud key yourself and choose to use it.

Every commit in the repository is signed, and every release artifact is signed and checksummed, so the thing you download is the thing we built.

Your thinking partner, on your machine.

